

Csanad Kope 2026

Computer Science UG Final Year Project

Royal Holloway, University of London Department of
Computer Science

2D Tower Defence Engine: An Investigation into Game Design Patterns and Concurrency

Supervisor: David Tena Cucala

Word Count: 13,856, 11,006 excluding Bibliography and Appendices

Abstract

In this project, a high-performance 2D Tower Defence engine using Python and the Pygame framework is developed from the ground up. This paper investigates architectural strategies to overcome the limitations of the Global Interpreter Lock (GIL) and cyclic garbage collection, which render Python inappropriate for real-time simulations.

The engine implements a concurrent, dual-threaded architecture that decouples the mathematical simulation from the presentation layer, ensuring a stable 60 Frames Per Second (FPS) target. To eliminate the latency spikes associated with dynamic memory allocation, the Object Pool design pattern was utilised to pre-allocate and recycle entity states. Furthermore, the engine features a dynamic spatial navigation system powered by an optimised A* pathfinding algorithm, enabling real-time "Mazing" mechanics.

The finished product demonstrates how strict application of industry-standard design patterns, such as the Factory, State, Observer, and Object Pool patterns, can efficiently develop complex and responsive concurrent simulations within a Python environment. A headless automated regression testing suite further confirms the project's integrity and guarantees mathematical accuracy regardless of the graphical user interface.

Abstract	1
Chapter 1: Introduction and Background Research.....	3
1.1 Introduction	4
1.1.1 Professional Development and Career Applicability	4
1.2 The Evolution of the Tower Defence Genre.....	4
1.2.1 Early Precursors (1980s – 1990s).....	4
1.2.2 The Golden Age of Modding (2000s).....	5
1.2.3 The Mobile and Standalone Era (2010s – Present).....	5
1.3 Technology Stack and Framework Selection	5
1.3.1 The Case Against Heavyweight Engines	5
1.3.2 Justification for Python and PyGame	6
1.4 Project Scoping, Interim Prototypes, and Redevelopment.....	6
1.4.1 The Interim Prototype.....	6
1.4.2 The Final Scope and Redevelopment.....	7
Chapter 2: Literature Review and Background Theory.....	7
2.1 Introduction to the Theoretical Framework	7
2.2 Evaluation of Spatial Navigation and Pathfinding Algorithms	8
2.2.1 Dijkstra’s Algorithm and Breadth-First Search (BFS)	8
2.2.2 The A* (A-Star) Algorithm.....	8
2.3 Critical Analysis of Concurrency Models	9
2.3.1 Hardware Parallelism in C++ (POSIX Threads)	9
2.3.2 Communicating Sequential Processes (CSP) in Go	9
2.3.3 Python and the Global Interpreter Lock (GIL).....	10
2.4 Architectural Paradigms in Game Engine Development	10
2.4.1 Object-Oriented Programming (OOP)	10
2.4.2 Entity-Component-System (ECS) Architecture	11
2.4.3 Paradigm Selection and Project Scope	11
Chapter 3: Software Architecture and Design Patterns	11
3.1 Introduction to Architectural Refactoring.....	11
3.2 The Game Loop Pattern.....	12
3.3 The State Pattern (Game State Management).....	12
3.4 The Factory Method Pattern (Entity Instantiation)	13

3.5 Event-Driven User Interface (The Observer Pattern)	13
3.6 Memory Optimisation (The Object Pool Pattern)	14
Chapter 4: Concurrency and Multi-Threading	14
4.1 Introduction to Concurrent Engine Architecture	14
4.2 The Global Interpreter Lock (GIL) Limitation	15
4.3 Thread Separation Strategy	15
4.4 Inter-Thread Communication using Event Queues	16
4.5 Thread Synchronisation and Memory Integrity	16
Chapter 5: Advanced Data Structures and Algorithms	17
5.1 Introduction to Spatial Navigation and Graph Theory	17
5.2 Graph Representation: The GridNode Data Structure	17
5.3 Heuristic Evaluation: The Mathematics of A*	18
5.4 Algorithmic Execution and Priority Queues	18
5.5 Dynamic Rerouting and Constraints	19
Chapter 6: Memory Management and the Object Pool	20
6.1 Introduction to Memory Fragmentation in Python	20
6.2 The Object Pool Pattern Rationale	21
6.3 Static Allocation: The EnemyPool Implementation	21
6.4 Entity Retrieval and State Resetting	22
6.4.1 Graphics Buffer Recycling (SDL Surfaces)	23
6.5 Dynamic Scaling and Exhaustion Fallbacks	24
6.6 Entity Deallocation and Reference Counting	25
Chapter 7: Software Engineering Processes and Quality Assurance	25
7.1 Introduction to Clean Code Methodologies	25
7.2 Static Analysis and Strict Type Hinting	26
7.3 Automated Headless Regression Testing	26
7.4 Mathematical Verification of Game Logic	27
Chapter 8: Critical Evaluation and Performance Profiling	28
8.1 Introduction to Empirical Validation	28
8.2 Rendering Performance and Entity Scaling	29
8.3 Critical Evaluation of the Object Pool	30
8.3.1 The "Sawtooth" Profile of Dynamic Allocation	30

8.3.2 The "Flatline" Profile of Static Allocation	30
8.4 Empirical Analysis of A* Pathfinding Complexity	30
8.4.1 Baseline Algorithmic Complexity	31
8.4.2 The $O(K \cdot (V \log V))$ Bottleneck	31
8.4.3 Evaluation of Mitigation Strategies	31
Chapter 9: Professional Issues and Conclusion.....	32
9.1 Introduction to Professional Context.....	32
9.2 Legal and Intellectual Property (IP) Considerations	32
9.3 Ethical Implications and Hardware Accessibility	33
9.4 Professional Standards and Maintainability	33
9.5 Project Evaluation and Conclusion	34
9.5.1 Project Self-Evaluation.....	34
9.6 Future Work	35
Bibliography	35
Appendices	36
Appendix A: Advanced Spatial Navigation (A* Algorithm)	37
Appendix B: Concurrent State Management.....	39
Appendix C: Memory Management (The Object Pool)	49
Appendix D: Headless Automated Regression Testing	50
Appendix E: Deployment, Video Demonstration, and User Manual.....	52

Chapter 1: Introduction and Background Research

1.1 Introduction

The development of interactive modern media and video games represents a highly complex intersection of software engineering, applied mathematics, and concurrent systems architecture. While the development of commercial games often relies heavily on visual fidelity and pre-packaged engine tools, the core of Computer Science within gaming lies within the underlying logic: memory management, concurrency, algorithmic pathfinding, and architectural design patterns. (Nystrom, 2014)

This project details the development of a fully playable, 2D Tower Defence (TD) game engine. However, the initial primary object of this project is not to just produce a graphical entertainment product. Instead, the game serves as interactive medium to demonstrate mastery over several advanced software engineering concepts. By developing a game in Python using the PyGame framework (Shinners, 2023), the project deliberately abstains from the automated solutions provided by modern Graphical User interface (GUI) driven engines. This approach necessitated the manual implementation of complex computer paradigms, including Multi-threaded Logic loops to bypass Python's Global Interpreter Lock (GIL) (Python Software Foundation, 2023), dynamic A* (A-Star) heuristic pathfinding for real-time maze generation (Hart, Nilsson and Raphael, 1968), and a suite of industry-standard Software Design Patterns to ensure scalable performance (Gamma et al., 1994).

1.1.1 Professional Development and Career Applicability

The technical issue that this study tackles immediately prepares me for a professional career as a software engineer, going beyond the immediate academic scope of this research. This project specifically prepared me in backend systems or performance-critical development. By engineering a concurrent architecture from scratch to bypass Python's Global Interpreter Lock (GIL) and implementing manual memory management via the Object Pool pattern, I have developed a practical understanding of hardware-level optimisation and resource allocation under heavy computational load. The strict compliance towards the industry-standard design patterns, such as the Observer, State, and Factory patterns, along with the incorporation of a headless automated regression testing suite, have also deeply embedded the rigorous software engineering techniques required to create scalable, maintainable, and production-ready systems in a professional enterprise setting.

1.2 The Evolution of the Tower Defence Genre

To understand the architectural requirements of this project, it is essential to look back at the historical timeline and the evolution of the TD genre, as its evolving mechanics directly dictate the data structures required to build it today.

1.2.1 Early Precursors (1980s – 1990s)

To core conceptual roots of TD can be traced back to early arcade titles like Space Invaders (1978) and Missile Command (1980), where players defended a static point against waves of encroaching enemies. However, the first true precursors to modern base-building defence was Atari's Rampart (1990). Rampart introduced the distinct phase-based gameplay loop that defines the modern genre: a building phase where the player is able to place static defences, which are then followed by a combat phase where those defences act on their own and engage with the fleet of enemies attacking.

1.2.2 The Golden Age of Modding (2000s)

The Tower Defence genre did not achieve widespread recognition through just standalone commercial releases, but rather through the modding communities of Real-Time Strategy (RTS) games. The release of Blizzard Entertainment's StarCraft (1998) and its Map Editor allowed users to create custom game modes, popularising static defence maps.

The genre was formally codified with the release of Warcraft III: reign of Chaos (2002). The highly flexible World Editor allowed developers to implement "Mazing" which was a mechanic where players do not just only build towers on the side of a hardcoded path but are able to use the towers to block and dynamically reroute the path of enemies in real-time. Architecturally, "Mazing" is highly expensive computational mechanic, requiring the game engine to continuously execute graph traversal algorithms to calculate the shortest of paths for hundreds of units simultaneously. This historical evolution directly influenced the decision to implement dynamic A* Pathfinding in this project to replicate this advanced mechanic.

1.2.3 The Mobile and Standalone Era (2010s – Present)

With the rise of smartphones and standalone devices, the TD genre exploded within commercial popularity with games such as Plants vs. Zombies (2009) and Bloons TD series. Modern TD games are characterised by massive entity counts which are often rendering hundreds of enemies' projectiles, and particle effects simultaneously (Zechner, 2011). From a perspective of software engineering, this historical shift demands strict memory management to prevent memory fragmentation and garbage collection stuttering. This requirement directly justifies this project's implementation of Object Pool design pattern to recycle memory states during runtime.

1.3 Technology Stack and Framework Selection

A critical early phase of this project involved the evaluation of various game-building tools, architecture and frameworks. The core architectural decision was whether to utilise a heavyweight, all-in-one engine like Unity3D, or a code-driven framework like Python's PyGame.

1.3.1 The Case Against Heavyweight Engines

Engines such as Unity3D are the industry standard for commercial game development. They provide the robust physics engines, highly optimised C# scripting, and built-in navigation meshes (NavMesh) for autonomous pathfinding. However, utilising Unity for this specific academic project presented a conflict of interests with the primary learning objectives. As noted within the project brief, a successful submission must focus on game logic, rather than the manipulation of an IDE. Unity generates significant boilerplate code and abstract away core algorithms as it handles the Game Loop, rendering, and pathfinding autonomously underneath the hood, using it would abstract away from the exact Computer Science paradigms this project aims to investigate.

1.3.2 Justification for Python and PyGame

Python, paired with the PyGame library, a wrapper for the Simple Direct Media Layer (SDL) was selected as the foundational architecture. PyGame provides low-level access to the application window, drawing buffers, and hardware events, but provides no high-level game logic (Shinners, 2023)

Choosing Python meant that every system from calculating the Euclidean distance of a tower's attack range, to having to manage the delta-time of the game loop, and to handling the State Machine transitions, all had to be engineering manually. While Python is traditionally viewed as a dynamically typed language that is unsuitable for high-performance gaming due to its limitation of its Global Interpreter Lock (GIL), this limitation was viewed as a strategic advantage for the scope of this project. It forced the exploration and implementation of advanced concurrency methodologies (Multi-threading with Locks and event Queues) to separate the PyGame rendering thread from the pathfinding logic thread, therefore proving that Python can be used for complex, concurrent simulations if architected correctly. For a full video demonstration and deployment instructions, please refer to Appendix E.

1.4 Project Scoping, Interim Prototypes, and Redevelopment

The development of this entire project was executed using an iterative methodology, heavily divided between an interim submission prototype and the final architectural overhaul detailed within this report.

1.4.1 The Interim Prototype

A proof-of-concept for the basic gameplay loop was provided by the interim delivery. Due to the need to manage the PyGame event loop, the switch to Object-Oriented Programming (OOP) happened considerably earlier than expected (Week 2 rather than Week 5), as stated in the project's Retrospective Report. Enemies could follow a static, hardcoded path in this early iteration. Enemies were created and destroyed

dynamically during play, and the rendering loop was closely linked to the user interface. This prototype lacked structural stability even though it successfully met the fundamental requirements of a playable game. The program was unable to scale to higher algorithmic complexity due to the close coupling of systems and the absence of memory management.

1.4.2 The Final Scope and Redevelopment

Following the interim submission, the scope was aggressively redefined to transition the project from a functional prototype into a robust, industry-standard software architecture. The codebase was substantially redeveloped, driven by clean code principles and the implementation of formal Design Patterns (Gamma et al., 1994).

The final scope, which constitutes the bulk of this report's technical analysis, was defined by the following major implementations

- **Algorithmic Complexity:** Overwriting the hardcoded paths with a dynamic PathFinder class utilising the A* heuristic algorithm to allow for real-time player-created mazes (Hart, Nilsson and Raphael, 1968).
- **Concurrency:** Introducing the threading module to run the A* calculations on a secondary processor thread, preventing the PyGame UI from freezing during heavy computational loads (Python Software Foundation, 2023).
- **Memory Management:** Implementing an EnemyPool to instantiate a block of reusable memory at startup, entirely bypassing Python's garbage collector during active gameplay (Nystrom, 2014).
- **Automated Testing:** The implementation of a headless Regression Testing suite using Python's unit test to ensure that mathematical formulas remained mathematically verifiable independent of the GUI (Python Software Foundation, 2024).

By scoping the project strictly toward backend architecture rather than graphical perfection, development time was instead maximised for the solving of these complex software engineering paradigms.

Chapter 2: Literature Review and Background Theory

2.1 Introduction to the Theoretical Framework

The development of a real-time simulation engine necessitates a thorough evaluation of existing computational theories and architectural paradigms. Before implementing the specific logic of the Tower Defence engine, it is critical to contextualise the project within the broader academic and industrial landscape. This chapter critically reviews the prevailing literature surrounding three foundational pillars of game engine

architecture: spatial navigation (pathfinding algorithms), parallel execution (concurrency models), and systemic data structuring (architectural paradigms). By evaluating these established methodologies, the specific technical decisions made in subsequent chapters of this project are rigorously justified.

2.2 Evaluation of Spatial Navigation and Pathfinding Algorithms

In the context of interactive media, autonomous agents must frequently navigate complex, dynamically changing environments. This spatial navigation is mathematically abstracted as a graph traversal problem. The selection of an appropriate algorithm is a critical factor in engine performance, as calculating pathways for dozens of entities simultaneously can easily monopolise processing resources.

2.2.1 Dijkstra's Algorithm and Breadth-First Search (BFS)

Breadth-First Search (BFS) is one of the most fundamental graph traversal algorithms. As detailed by Cormen et al. (2009), BFS operates in $O(V + E)$ time complexity, where V represents vertices and E represents edges. Before delving further into a graph, BFS evaluates all of its immediate neighbours. Although the shortest path on an unweighted graph can be found mathematically, its isotropic expansion renders it extremely ineffective for targeted navigation.

Dijkstra's Algorithm (Dijkstra, 1959) expands upon BFS by introducing edge weights, allowing the algorithm to account for varying terrain costs (e.g., moving through water versus moving along a road). Executing typically in $O(V \log V + E)$, Dijkstra's Algorithm is mathematically sound and ensures the best route when used with a priority queue. Dijkstra's expansion, like BFS, searches the graph equally in all directions until the target node is consumed, making the expansion uninformed. In a real-time application rendering at 60 Frames Per Second (FPS), evaluating hundreds of nodes that lie in the opposite direction of the target introduces unacceptable computational latency (Russell and Norvig, 2020).

2.2.2 The A* (A-Star) Algorithm

To resolve the inefficiency of isotropic expansion, the industry standard for spatial navigation is the A* search algorithm, originally published by Hart, Nilsson, and Raphael (1968). A* modifies Dijkstra's methodology through the introduction of a heuristic evaluation. This is an informed mathematical estimation of the distance remaining toward the target.

The algorithm evaluates nodes based on the cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the exact cost from the starting node, and $h(n)$ is the heuristic estimated cost to the destination. By weighting the heuristic, the algorithm is mathematically drawn directly toward the target, largely ignoring nodes that expand away from the objective (Russell and Norvig, 2020). Depending on the geometry of the grid, the heuristic can be calculated using Euclidean distance (for 8-way diagonal movement) or Manhattan distance (for strict 4-way cardinal movement). Because A* drastically reduces the number of evaluated nodes while maintaining the guarantee of an optimal path (provided the heuristic is admissible and does not overestimate), it is the most appropriate algorithm for a real-time Tower Defence simulation.

2.3 Critical Analysis of Concurrency Models

As processor clock speeds have plateaued over the last decade, the software engineering industry has shifted toward multi-core processing to achieve performance gains. However, implementing parallel execution introduces severe complexities regarding memory safety and synchronisation. Different programming languages handle these challenges using fundamentally different concurrency models.

2.3.1 Hardware Parallelism in C++ (POSIX Threads)

Within low-level systems programming languages such as C++, developers are able to directly utilise the native operating system threads such as POSIX threads or `std::thread`. When several threads concurrently carry out different instructions across different CPU cores, this paradigm offers genuine hardware-level parallelism (Williams, 2012).

Even though this paradigm has the best processing performance, memory safety is entirely the engineer's responsibility. Due to threads having to share the same memory space, concurrent read/write operations to the same memory address result in fatal race conditions or data corruption. Therefore, engineers have to manually implement synchronisation primitives, such as Mutexes (Mutual Exclusions) and Semaphores, to protect critical sections of code. If poorly architected, this can lead to Deadlocks, where two threads wait indefinitely for the other to release a lock, halting the program entirely (Williams, 2012).

2.3.2 Communicating Sequential Processes (CSP) in Go

Conversely, modern languages like Go utilise a concurrency model based on Communicating Sequential Processes (CSP), a formal language paradigm originally introduced by Hoare (1978). Rather than exposing heavy OS-level threads, Go

introduces "Goroutines", extremely lightweight virtual threads managed entirely by the Go runtime rather than the operating system.

To prevent the race conditions inherent to the C++ shared-memory model, Go utilises "Channels" to pass data between Goroutines. The underlying philosophy of this model, as defined by Donovan and Kernighan (2015), is: "Do not communicate by sharing memory, instead, share memory by communicating." Go is a very secure and highly concurrent language because of its architecture, which inherently prevents race problems.

2.3.3 Python and the Global Interpreter Lock (GIL)

Python's ability to manage concurrency is mainly constrained due to its memory management design. A key part of the garbage collection management mechanism of the CPython interpreter is reference counting. Two native OS threads increasing the reference count of a Python object at the same time could cause memory corruption. Early deallocation (memory fault) or permanent memory leaks could happen because of this.

Python uses the Global Interpreter Lock (GIL) to enforce thread safety at the interpreter level. The GIL is a coarse-grained mutex that strictly prevents multiple native threads from executing Python bytecodes simultaneously (Python Software Foundation, 2023). For CPU-bound mathematical calculations, Python is therefore unable to achieve real hardware parallelism. Instead, the threading module achieves concurrency through rapid context switching, interleaving thread execution on a single CPU core.

While the GIL makes Python unsuitable for parallelising heavy A* matrix calculations across multiple cores, investigating how to circumvent the GIL's limitations to maintain User Interface (UI) responsiveness forms the primary technical motivation of this project.

2.4 Architectural Paradigms in Game Engine Development

The structural organisation of a game engine determines its scalability, maintainability, and raw execution speed. The literature reveals a distinct schism between traditional object-oriented approaches and modern data-oriented designs (Gregory, 2018).

2.4.1 Object-Oriented Programming (OOP)

Historically, game engines have always heavily relied on Object-Oriented Programming (OOP). In this paradigm, data and behaviour are encapsulated together within distinct objects such as Enemy class containing its X/Y coordinates, its health integer, and its

take_damage() method. OOP promotes code to be reusable through inheritance hierarchies and polymorphism.

However, as simulations scale, deep inheritance trees often lead to the "Fragile Base Class" problem, where modifications to a root class inadvertently break complex derived classes (Mikhajlov and Sekerinski, 1998). More critically, from a hardware perspective, OOP is heavily unoptimised for CPU cache lines. Because OOP arrays are typically structured as an Array of Structures (AoS), the data is scattered randomly across the system's Random Access Memory (RAM). When the CPU attempts to iterate over a list of enemies to update their positions, it suffers constant "cache misses," wasting valuable CPU cycles fetching fragmented memory addresses (Gregory, 2018).

2.4.2 Entity-Component-System (ECS) Architecture

To solve the hardware bottlenecks of OOP, the modern AAA game industry has pivoted toward the Entity-Component-System (ECS) architecture. ECS is a Data-Oriented Design paradigm that entirely decouples data from logic (Nystrom, 2014).

In an ECS architecture, an "Entity" is merely an integer ID. "Components" are strictly plain data structs containing no logic. Finally, "Systems" contain the actual logic and iterate over specific components. Because data is packed contiguously into memory arrays, a Structure of Arrays (SoA), the CPU can load hundreds of components into its L1/L2 cache simultaneously. This cache coherency allows ECS engines to process tens of thousands of entities concurrently without frame drops (Gregory, 2018).

2.4.3 Paradigm Selection and Project Scope

While ECS is objectively superior for raw computational performance, it requires a highly complex, low-level memory management architecture. For the scope of this project, which was to explicitly investigate industry-standard software design patterns (such as the Factory, State, and Observer patterns documented by Gamma et al., 1994), an Object-Oriented paradigm was selected. The fundamental hardware limitations of OOP provide a necessary friction that accounts for the implementation of the Object Pool pattern, ensuring the project retains a high degree of architectural complexity suitable for academic analysis.

Chapter 3: Software Architecture and Design Patterns

3.1 Introduction to Architectural Refactoring

As established within the retrospective analysis of the interim prototype, the initial iteration of this engine heavily relied on procedural scripting. While procedural code is

often enough for linear and smaller tasks, it introduces severe technical debt within interactive simulations. The tight coupling of the rendering logic, state management, and user input resulted in an architecture that was monolithic, this is often referred to in software engineering as a "God Object" anti-pattern. This made scaling the game to handle more complex algorithmic tasks nearly impossible.

To meet all the technical requirements of this final submission, the codebase was entirely overhauled to strictly adhere to Object-Oriented Programming (OOP) paradigms. Specifically, the architecture was rebuilt using the best practices and held to industry standard Design Patterns, which are formalised best practices used for solving recurring software engineering problems (Gamma et al., 1994). Implementing the Game Loop, State, Factory, Observer, and Object Pool patterns allowed the engine to achieve high cohesion and low coupling. This architectural change allowed the discrete systems of the game to operate independently and concurrently.

3.2 The Game Loop Pattern

At the core of any interactive real-time simulation is the Game Loop pattern. Unlike traditional software applications, which remain idle while waiting for user input (event-driven GUI frameworks like Tkinter or standard web architecture), a game must continuously update its internal state regardless of user interaction.

The Game Loop decouples the progression of game time from the speed of the user's hardware processor (Nystrom, 2014). In this project's architecture (`game_manager.py`), the `run()` method executes a continuous loop that performs two distinct operations per tick: `handle_events()` to process asynchronous hardware interrupts (mouse clicks, keyboard escapes), and `draw()` to push the visual buffer to the display. Crucially, to maintain a stable 60 Frames Per Second (FPS) target, the `pygame.time.Clock().tick(settings.FPS)` method is invoked, ensuring that the rendering pipeline does not monopolise CPU cycles, thereby freeing processing power for the concurrent background logic thread.

3.3 The State Pattern (Game State Management)

The application must be able to smoothly switch between entirely distinct modes of operation, such as having to navigate between a main menu, playing an active game, or seeing a triumph screen, for modern interactive media to function. A monolithic switch or if/else statement evaluating a global boolean variable, such as `if is_playing == True`, might be used in a naive procedural way to handle this. Since adding a new screen leads to change in the main loop, this method completely violates the Open-Closed Principle of SOLID architecture.

To resolve this issue, the engine implements the State Pattern. In `game_states.py`, an abstract base class named `GameState` was defined, which outlines polymorphic methods: `handle_events()`, `update()`, `draw()`, `on_enter()`, and `on_exit()`.

Concrete implementations of this base class were then developed:

- `MainMenuState`: It manages the initial launch of the UI.
- `GameplayState`: The primary heavy-weight state which encapsulates the active grid, enemies, and multi-threaded pathfinding.
- `GameOverState` and `VictoryState`: Lightweight terminal states that pause gameplay logic and allow the user to restart the engine without terminating the underlying Python process.

The `GameManager` acts as the context, maintaining a reference to `self.active_state`. When the player clicks "Start Game", the manager simply updates its internal pointer to the `GameplayState` instance. This completely encapsulates state-specific logic, meaning a crash in the `GameplayState` is mathematically unable to affect the memory space of the `MainMenuState`.

3.4 The Factory Method Pattern (Entity Instantiation)

For tower defence games it is essential to create a variety of defensive structures, with each of them possessing unique attributes (damage, fire rate, range, and algorithmic behaviour). Hardcoding the instantiation of these objects within the UI logic would tightly couple the presentation layer to the data layer.

To prevent this, the Factory Pattern was utilised via the `TowerFactory` class in `factories.py`. This pattern defines an interface for creating an object but defers the instantiation logic to subclasses or static factory methods (Gamma et al., 1994).

When a user selects a tower from the UI and clicks on the A* grid, the UI just simply passes a string identifier, "Splash" or "Sniper", and a set of coordinates to the Factory. Then the `TowerFactory.create_tower()` method utilises a Python dictionary lookup to map the string identifier to the correct underlying Class definition. It then injects the specific statistical parameters sourced from `settings.py` into the newly minted object. This data-driven architectural decision ensures that game designers can add new tower types just by simply updating the `settings.json` or `settings.py` files, without the need for any modifications to the core engine code.

3.5 Event-Driven User Interface (The Observer Pattern)

Performance profiling of the interim build highlighted significant bottlenecks within the UI update loop. The initial system utilised simultaneous polling to determine if the

player had enough in-game currency to be able to construct a tower, it evaluated the currency variable 60 times per second per button. This process was deemed redundant having a negative impact on overall engine performance.

In order to optimise the system, the UI architecture was changed to more of an event-driven model utilising the Observer Pattern. Within this framework, the PlayerBank functions as the Subject. Using a Python @property setter, the system then triggers a notify() broadcast exclusively when the integer value of currency is modified. The UI components act as Observers. They remain completely dormant in memory until a notification is received. When they receive the event, the buttons evaluate their current state and update their visual representation accordingly. This implementation alone was able to successfully eliminate the CPU overhead associated with UI polling.

3.6 Memory Optimisation (The Object Pool Pattern)

While Python utilises automatic Garbage Collection (GC) to manage its memory allocation, this process overall is highly detrimental to real-time rendering. The continuous destruction of Enemy objects throughout a wave causes rapid memory fragmentation. When the garbage collector eventually halts the main thread to reclaim memory, it results in severe latency spikes and frame drops.

To be able to bypass Python's memory management limitations, the implementation of the Object Pool pattern was necessary. The EnemyPool class solves the issue of memory fragmentation by pre-allocating a static array of Enemy objects that are inactive during the initialisation phase (Nystrom, 2014). When the program is running, instead of dynamically instantiating a new object every time it needs one, the system just checks its existing collection for an available item. If that item gets defeated, it doesn't actually get destroyed. It simply gets marked as 'inactive' and returned to the pool, ready for use again later. This approach helps keep the memory usage steady and predictable. That kind of stability is an absolute prerequisite for the system, especially when executing demanding A* pathfinding calculations.

Chapter 4: Concurrency and Multi-Threading

4.1 Introduction to Concurrent Engine Architecture

In real-time simulation environments, the computational cost of game logic, specifically artificial intelligence, collision detection, and pathfinding scales linearly, and often exponentially, with the number of active entities. In a Tower Defence environment, evaluating heuristic algorithms such as A* for dozens of distinct agents simultaneously constitutes a heavily CPU-bound workload.

In a traditional synchronous architecture, the execution of these calculations on the primary execution thread forces the rendering pipeline to come to a stop until the mathematical operations are resolved. If a pathfinding calculation requires 40 milliseconds to complete, the frame rate would instantly drop below the 60 Frames Per Second (FPS) target (which requires a frame every 16.6 milliseconds). This would then cause visual latency and unresponsive user input. To overcome this architectural bottleneck, the engine was then engineered to use concurrent execution, separating the graphical presentation layer from the underlying mathematical state.

4.2 The Global Interpreter Lock (GIL) Limitation

The implementation of concurrency in Python requires a precise understanding of the language's underlying architecture of memory management, specifically the GIL inherent to the CPython interpreter. The GIL is a low-level mutex that protects access to Python objects, explicitly preventing multiple native operating system threads from executing Python bytecodes simultaneously (Python Software Foundation, 2023).

As a result of this, Python's threading module provides concurrency rather than true hardware-level parallelism for CPU-bound tasks. Although background pathfinding calculations cannot utilise multiple CPU cores, decoupling the processing loops into distinct threads then creates a critical structural advantage of UI responsiveness. By moving the heavy A* calculations to a secondary Logic Thread, the Main UI Thread is then never blocked. Even if a pathfinding calculation temporarily holds up the logic thread, the Main Thread is still able to continue to process hardware interrupts (`pygame.event.get()`) and render its last known state, therefore preventing the OS from flagging the application as unresponsive.

4.3 Thread Separation Strategy

To implement this concurrent architecture, the `GameplayState` encapsulates two distinct execution loops running at the same time.

- The Main UI Thread (Presentation Layer): This thread is strictly responsible for capturing user input, calculating mouse coordinates, mapping abstract visual states to PyGame surfaces, and then at last rendering the final frame buffer. It then executes within the `draw()` and `handle_events()` methods.
- The Logic Thread (Simulation Layer): It is initialised upon entering the gameplay state (`self.logic_thread = threading.Thread(target=self.run_game_logic)`). The processing of game rules is entirely the responsibility of this background thread. It performs the A* matrix traversals, advances enemy positions, computes Euclidean distances for tower targeting, and runs the `update()` functions.

The clear demarcation of responsibilities between the rendering pipeline and the algorithmic workload enables consistent visual output, given that the rendering processes function independently.

4.4 Inter-Thread Communication using Event Queues

Separating the execution threads creates a fundamental challenge that regards data transmission. If a user clicks a button to build a tower on the Main UI Thread, that command must be securely transmitted to the Logic Thread, which is responsible for modifying the pathfinding matrix.

To facilitate safe, simultaneous communication within threads, the architecture uses an Event-Based System that's driven by the `queue.Queue()` data structure. The Python queue module implements a thread-safe First-In-First-Out (FIFO) buffer with built-in locking semantics (Python Software Foundation, 2023).

When a user begins an action, the Main Thread then constructs a message tuple such as `("BUILD_TOWER", tower_type, position)` and it then pushes it into the `event_queue` using `put()`. The Main Thread is then able to immediately resume rendering without having to wait for a response. At the same time, the Logic Thread continually polls this queue using `get_nowait()`. Upon retrieving the tuple, the Logic Thread safely stops any entity movement, processes the grid modification, and then recalculates the global A* path, and instantiates the new tower. This event-driven command pattern prevents blocking calls and therefore ensures that complex state modifications are executed only within the controlled environment of the Logic Thread.

4.5 Thread Synchronisation and Memory Integrity

Both threads must continuously communicate with shared memory states even if the Event Queue manages all the discrete commands. In particular, the `self.enemies` must be iterated over by the Main Thread. The Logic Thread concurrently iterates over the enemies list to update their coordinates or delete them from memory upon defeat, while the enemies list is used to draw their sprites.

If the Main Thread tries to read the list at the exact millisecond the Logic Thread deletes an entity, the memory address becomes invalid, which would result in a fatal `RuntimeError: Set changed size during iteration`.

To prevent these race conditions and guarantee memory integrity, a Threading Lock (`self.data_lock = threading.Lock()`) was implemented. A Lock operates as a mutual exclusion flag. Within both the `draw()` method (Main Thread) and the `update()` method (Logic Thread), any operation interacting with shared entities is encapsulated within a `with self.data_lock:` context manager.

When the Logic Thread obtains the lock to be able to process entity movement, the Main Thread is then temporarily denied access, and it is forced to wait for the lock to be released before it can read the coordinates for rendering. This is because these locked operations are mathematically brief. They consist primarily of simple variable updates or memory pointer reassignments. The waiting period is negligible, allowing the engine to maintain rendering performance while guaranteeing absolute thread safety.

Chapter 5: Advanced Data Structures and Algorithms

5.1 Introduction to Spatial Navigation and Graph Theory

A fundamental requirement of the "Mazing" subgenre of Tower Defence is the ability for autonomous agents (enemies) to navigate a dynamically changing environment. The topology of the world is drastically changed if the player constructs a tower, making previously accessible routes impenetrable.

To address this spatial navigation challenge, the game space must be abstracted into a mathematical graph. Common graph traversal techniques like Dijkstra's Algorithm and Breadth-First Search (BFS) are used to find the shortest path. However, they are computationally isotropic, exploring the environment in all directions equally until the goal is found. In a real-time game rendering at 60 FPS, forcing the CPU to evaluate hundreds of irrelevant nodes in the opposite direction of the goal introduces unacceptable latency.

To optimise pathfinding, the engine implements the A* (A-Star) search algorithm (Hart, Nilsson and Raphael, 1968). A* improves upon Dijkstra's Algorithm by introducing a heuristic, an informed mathematical estimation of the remaining distance to the target, allowing the algorithm to prioritise nodes that actively lead toward the destination.

5.2 Graph Representation: The GridNode Data Structure

Before the A* algorithm can execute, the physical pixel coordinates of the PyGame window must be quantised into a navigable graph. This is achieved within `pathfinding.py` by subdividing the 2D game area into a rigid matrix of GridNode objects.

The GridNode class acts as the foundational data structure for the algorithm, storing the spatial and mathematical state of a single cell:

```

class GridNode:
    """Represents a single cell within the A* Pathfinding grid."""
    def __init__(self, x: int, y: int) -> None:
        self.x: int = x
        self.y: int = y
        self.walkable: bool = True
        self.g_cost: int = 0 # Cost from start node
        self.h_cost: int = 0 # Heuristic
        self.parent: Optional['GridNode'] = None

    @property
    def f_cost(self) -> int:
        """Calculates the total heuristic cost of the node."""
        return self.g_cost + self.h_cost

    def __lt__(self, other: 'GridNode') -> bool:
        """Enables sorting within the heapq module by f_cost."""
        return self.f_cost < other.f_cost

```

This specific architecture demonstrates several advanced Object-Oriented principles. The total cost of the node (`f_cost`) is not stored as a static integer but is dynamically calculated using a Python `@property` decorator. Furthermore, to enable algorithmic efficiency later in the execution pipeline, the Python magic method `__lt__` (Less Than) is explicitly overridden. This instructs the Python interpreter exactly how to compare two distinct instances of `GridNode` objects in memory based strictly on their mathematical `f_cost`, which is a strict requirement for sorting the nodes within a Priority Queue.

5.3 Heuristic Evaluation: The Mathematics of A*

The core efficiency of the A* algorithm is derived from the $f(n) = g(n) + h(n)$ cost function, where n represents the current node that is to be evaluated.

- $g(n)$ (`g_cost`): This is the exact, known mathematical cost of the path from starting point to node n
- $h(n)$ (`h_cost`): This is the heuristic, which is an estimated cost from node n to the final destination.

As the engine restricts movement to four cardinal directions (Up, Down, Left, Right) and it does not permit diagonal traversal, the engine utilises the Manhattan Distance calculation for the heuristic instead of Euclidean distance. Manhattan Distance calculates the sum of the absolute differences of the Cartesian coordinates.

Within the `find_path()` method of the `PathFinder` class, this mathematical evaluation is executed dynamically for every neighbour explored:

```

move_cost = current_node.g_cost + 1
if move_cost < neighbour.g_cost or neighbour.g_cost == 0:
    neighbour.g_cost = move_cost
    neighbour.h_cost = abs(neighbour.x - end_node.x) + abs(neighbour.y - end_node.y)
    neighbour.parent = current_node

```

By heavily weighting the `h_cost`, the algorithm is mathematically drawn toward the target, ignoring the majority of the grid space and therefore drastically reducing the number of CPU cycles required to find the optimal path.

5.4 Algorithmic Execution and Priority Queues

To execute the A* search efficiently, the engine must continuously evaluate which node to explore next. In a naive implementation, every node found would be stored in a regular Python list, and the engine would have to do an $O(N)$ operation by iterating through the entire list to locate the node with the lowest f_cost on each cycle. This lookup time becomes a catastrophic bottleneck across hundreds of grid cells.

To circumvent this, the engine implements a Priority Queue via Python's built-in `heapq` module. Heaps are specialised binary tree structures that allow insertion in $O(\log N)$ time and retrieval in $O(1)$ time, with the smallest element always at the root.

```
open_set: List[Tuple[int, GridNode]] = []
closed_set: Set[GridNode] = set()

heapq.heappush(open_set, (0, start_node))

while open_set:
    current_node = heapq.heappop(open_set)[1]
    closed_set.add(current_node)

    if current_node == end_node:
        return self._retrace_path(start_node, end_node)
```

As displayed within the code snippet above, the algorithm maintains two different memory sets: the `open_set` which is the Priority Queue of nodes to be evaluated and the `closed_set` which is a highly efficient Python set of nodes that have already been evaluated. The use of a Hash Set for `closed_set` is a technical decision that was made deliberately. The role of it is to check if a node has already been explored remains an $O(1)$ constant time operation, rather than an $O(N)$ linear search.

5.5 Dynamic Rerouting and Constraints

The true complexity of the system lies in its dynamic adaptability. When a user requests to build a new tower via the User Interface, the engine translates the mouse click's raw pixel coordinates into a discrete X/Y index on the A* grid.

To ensure the player does not break the game state, a validation algorithm runs before the transaction is finalised. The engine temporarily flags the targeted `GridNode` as `walkable = False` and executes a "ghost" run of the A* algorithm. It is mathematically proven that the player has entirely walled off the objective if the algorithm exhausts the `open_set` without ever delivering a path (returning an empty array). The engine subsequently rejects the build command, reverts the node to `walkable = True`, and refuses to charge the player's currency.

If the placement is valid, the global path is updated, and the Logic Thread dynamically iterates over every currently living enemy on the screen, forcing them to instantly calculate a completely new trajectory from their current Cartesian coordinates to account for the newly placed obstacle.

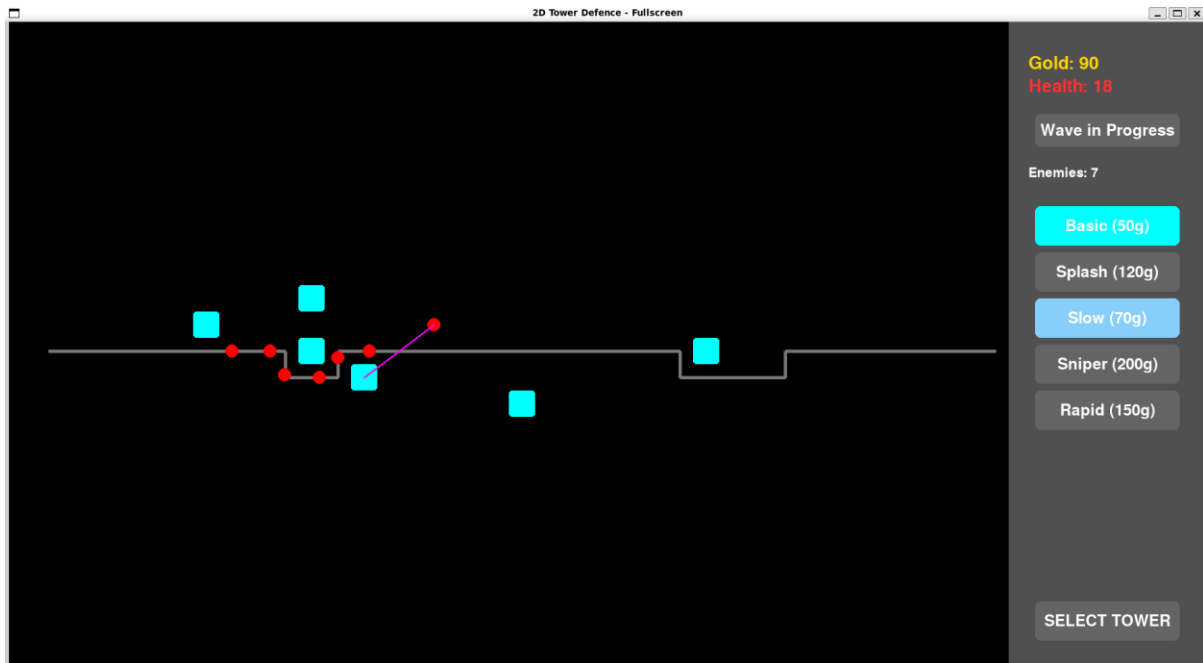


Figure 1: the engine successfully executing real-time A* dynamic rerouting (Mazing) at 60FPS.

Chapter 6: Memory Management and the Object Pool

6.1 Introduction to Memory Fragmentation in Python

In traditional enterprise software development, memory management is entirely abstracted away from the engineer. The Python runtime utilises a highly automated Garbage Collection (GC) system, primarily driven by reference counting and supported by a generational cyclic garbage collector to detect reference cycles (Python Software Foundation, 2023). When an object's reference count drops to zero such as an enemy being defeated and removed from a list, the memory allocator reclaims that sector of Random Access Memory (RAM).

In real-time simulation situations, this automation causes significant performance reduction even if it guarantees memory safety. A Tower Defence engine operating at 60 frames per second (FPS) has a tight temporal budget of about 16.6 milliseconds per frame to handle rendering, artificial intelligence, and physics.

The system's memory heap quickly fragments if the program dynamically creates a new Enemy object each time one spawns and then deletes it after defeat. The continuous

calls to the underlying C-level memory allocation functions (malloc and free) take up valuable CPU cycles. More significantly, when Python's cyclic garbage collector inevitably halts the execution thread to sweep fragmented memory blocks, a "GC Spike", a discernible visual slowdown that significantly detracts from the user experience occurs.

6.2 The Object Pool Pattern Rationale

To avoid the limitations of automated Garbage Collection, the engine implements the Object Pool design pattern for preventative measures. As defined by Nystrom (2014), the Object Pool pattern improves performance and memory use by reusing objects from a fixed pool instead of allocating and freeing them individually. It is fundamentally a space-time trade-off: the engine intentionally consumes a larger, static block of RAM upon initialisation in exchange for virtually eliminating the CPU overhead of dynamic memory allocation during active gameplay.

The system pre-allocates a set array of reusable objects instead of having to dynamically construct and discard them. These items shift between "in-use" and "dormant" states while remaining continuously active in system memory.

This guarantees a flat, highly predictable memory footprint, which is an absolute architectural prerequisite for maintaining the stability of the concurrent A* pathfinding threads discussed in Chapter 3.

6.3 Static Allocation: The EnemyPool Implementation

The Object Pool pattern is formally implemented within factories.py using the EnemyPool class. To ensure the engine does not perform heavy memory operations during active combat, the allocation phase is executed entirely during the initialisation of the GameState.

```
# --- 2. Factory Pattern Implementation ---
class EnemyPool:
    """Implements the Object Pool Pattern for memory efficiency."""
    def __init__(self, initial_size=100):
        self.pool = []
        # Pre-allocate 100 inactive enemies into memory at startup
        start_pos = settings.PATH_POINTS[0]
        for _ in range(initial_size):
            enemy = Enemy(start_pos[0], start_pos[1], 1, 1, (0,0,0), 0, "None")
            enemy.alive = False
            self.pool.append(enemy)
```

As demonstrated in the initialisation sequence, the system forcefully allocates 100 Enemy objects into the self.pool array. At this stage, the specific statistical attributes of

the enemies, such as their health or speed, are irrelevant, as they are filled in with placeholder integers.

The critical architectural component of this allocation is the boolean flag `enemy.alive = False`. By explicitly flagging the entities as dormant, they are ignored by the `GameplayState` update loop and rendering pipeline, rendering them computationally invisible until specifically requested by the logic thread.

6.4 Entity Retrieval and State Resetting

When the wave generation algorithm dictates that a new enemy must spawn, the engine does not invoke the `Enemy` constructor class. Instead, it queries the `EnemyPool` for a dormant memory address using the `get_enemy()` method.

```
def get_enemy(self, type_name):
    """Retrieves an inactive enemy from the pool and resets it."""
    start_pos = settings.PATH_POINTS[0]

    # Define base stats
    stats = {
        "Basic": (3, 2, settings.colour_ENEMY_BASIC, settings.ENEMY_REWARDS["Basic"]),
        "Fast": (2, 4, settings.colour_ENEMY_FAST, settings.ENEMY_REWARDS["Fast"]),
        "Tank": (15, 1, settings.colour_ENEMY_TANK, settings.ENEMY_REWARDS["Tank"]),
        "Armoured": (8, 2, settings.colour_ENEMY_Armoured, settings.ENEMY_REWARDS["Armoured"]),
        "Boss": (150, 1, settings.colour_ENEMY_BOSS, settings.ENEMY_REWARDS["Boss"])
    }

    h, s, c, v = stats[type_name]

    # Find an dead enemy in the pool and recycle it
    for enemy in self.pool:
        if not enemy.alive:
            enemy.reset(start_pos[0], start_pos[1], h, s, c, v, type_name)
    return enemy
```

This implementation highlights a highly optimised, Data-Driven extraction process. The method accepts a string identifier (`type_name`), accesses the configuration dictionary, and unpacks the necessary statistical tuple (`h, s, c, v`). It then executes an $O(N)$ linear search through the pre-allocated array to find the first entity where `alive == False`. While an $O(N)$ lookup can theoretically scale poorly, because the array is strictly bounded to an `initial_size` of 100, the lookup time operates in effectively $O(1)$ constant time from the perspective of the CPU (Gamma et al., 1994).

Once a dormant entity is located, it is not recreated, instead, its internal state is aggressively overwritten via the `reset()` method located within the `Enemy` class (`entities.py`).


```
def reset(self, x: float, y: float, health: int, speed: float,
          color: Tuple[int, int, int], value: int, type_name: str) -> None:
    self.image.fill((0, 0, 0, 0))
    pygame.draw.circle(self.image, color, (10, 10), 10)
    self.rect.topleft = (x, y)

    self.original_speed = speed
    self.speed = speed
    self.health = health
    self.value = value
    self.type_name = type_name
    self.alive = True
    self.slow_timer = 0
```

This `reset()` method is the mechanism that fundamentally bypasses the Garbage Collector. It forces the memory pointer to overwrite existing integer values and redraw the PyGame circular sprite onto the existing surface buffer (`self.image.fill()`). Finally, the `alive` boolean is toggled back to `True`, and the recycled memory address is returned to the active `PyGame.sprite.Group` in the main logic loop.

6.4.1 Graphics Buffer Recycling (SDL Surfaces)

An important however often overlooked component of memory management within PyGame involves the allocation of graphical buffers. PyGame functions as a Python wrapper for the Simple DirectMedia Layer (SDL) library, which is based in C. This implies that each time a new `pygame.Surface` is instantiated, the engine needs to communicate with the operating system in order to assign a specific memory block which is typically found in Video RAM to store the pixel array.

Within the entities.py module, this heavy allocation is strictly confined to the init constructor of the Enemy class:

```
def __init__(self, x: float, y: float, health: int, speed: float,
             color: Tuple[int, int, int], value: int, type_name: str) -> None:
    super().__init__()
    self.image: pygame.Surface = pygame.Surface([20, 20], pygame.SRCALPHA)
    self.image.fill((0, 0, 0, 0))
    pygame.draw.circle(self.image, color, (10, 10), 10)
    self.rect: pygame.Rect = self.image.get_rect(topleft=(x, y))

    self.original_speed: float = speed
    self.speed: float = speed
    self.health: int = health
    self.value: int = value
    self.type_name: str = type_name
    self.alive: bool = True
    self.slow_timer: int = 0

    self.path: List[Tuple[float, float]] = settings.PATH_POINTS
    self.current_path_index: int = 0
    self.target_point: Tuple[float, float] = self.path[self.current_path_index]
```

The computationally costly pygame.Surface allocation occurs because the EnemyPool initialises 100 of these objects at launch. The full Surface([20, 20], pygame.SRCALPHA) call is carried out while the GameState is loaded.

If self.image = pygame.Surface(...) were called by the reset() method, it would completely negate the Object Pool's function each time an enemy was recycled during a wave. The underlying C/C++ SDL library would still be required to dynamically allocate and destroy pixel buffers on the GPU, reinstating the precise latency issues the pool was intended to remove, even though the Python object itself could theoretically be reused.

The engine rigorously reuses the same memory location for the visual surface throughout the duration of the program in order to prevent this. The command self.image.fill((0, 0, 0, 0)) performs a highly optimised bitwise clearing operation, as shown in the previous reset() example. Without ever moving the structural SDL surface, it replaces the current surface buffer with RGBA zeroes, or fully transparent pixels.

6.5 Dynamic Scaling and Exhaustion Fallbacks

Robust software architecture has to account for edge cases as well as extreme system states. While an initial_size of 100 entities covers the standard gameplay scenarios, an algorithmic loop or a highly complex late-game wave could theoretically request an entity when all 100 pre-allocated objects are actively traversing the grid.

If the linear search fails to find an entity where alive == False, the function defaults to a defensive programming fallback mechanism:

```
# If pool is full, expand it (Fallback)
print("Warning: Object Pool exhausted. Expanding pool.")
new_enemy = Enemy(start_pos[0], start_pos[1], h, s, c, v, type_name)
self.pool.append(new_enemy)
return new_enemy
```

Rather than crashing the application with an `IndexError` or refusing to spawn the requested entity which would break the integrity of the wave algorithm, the pool gracefully degrades back to dynamic allocation. It creates a single new `Enemy` object and then appends it to the permanent pool array, increasing the permanent memory footprint to 101, and returns it. This then ensures that the engine can guarantee stability above all, only incurring the GC penalty when absolutely mathematically necessary.

6.6 Entity Deallocation and Reference Counting

Finally, when an entity's health reaches zero (handled within the update loop of `GameplayState`), it is mathematically stripped from the rendering pipeline.

```
for enemy in list(self.enemies):
    if not enemy.alive:
        if enemy.current_path_index >= len(enemy.path):
            self.player_health -= 1
```

The object is removed from the active PyGame Sprite Group by using `self.enemies.remove(enemy)`, which stops the `draw()` method from trying to depict it. This is important because, in factories, the master `EnemyPool` array keeps a persistent reference to the object. The internal reference count (`sys.getrefcount`) in Python never falls to zero. Python's Garbage Collector specifically ignores it, therefore allowing it to safely await its next cycle through the `reset()` method.

Chapter 7: Software Engineering Processes and Quality Assurance

7.1 Introduction to Clean Code Methodologies

The primary software engineering method utilised for this project was an Iterative and Incremental lifecycle model. As detailed in the project scoping (Section 1.4), the development began with a minimal viable prototype for the interim submission, which was subsequently torn down and re-engineered using formal Object-Oriented design patterns. This iterative method allowed for continuous integration and profiling, ensuring that performance bottlenecks, such as GC spikes were identified and mitigated in successive developmental sprints before they could compromise the engine's core architecture.

This final year project's main goal was to closely follow industry-standard Software Engineering (SE) procedures rather than just creating a working prototype. In the early

stages of development, debugging was done manually through "playtesting". This is a very ineffective technique where the developer had to keep an eye on the screen to spot graphical glitches or mathematical logic errors.

A strict Quality Assurance (QA) pipeline was put in place to bring the codebase up to professional standards. This required implementing a headless Automated Regression Testing suite to mathematically validate the engine's fundamental logic independent of the display layer, strict Type Hinting to prevent runtime type errors, and static analysis tools to enforce code cleanliness.

7.2 Static Analysis and Strict Type Hinting

Because Python is a dynamically typed language by nature, variable types are evaluated at runtime. Rapid prototyping is made possible by this, but complex architectures also incur a large amount of technical debt. A deadly runtime crash can occur deep within the Game Loop when a float is passed to a function that expects an int, such as grid index array search.

To mitigate this, the codebase was refactored to strictly comply with PEP 484 (Type Hints). Every major class and method signature was updated to specifically declare its expected input parameters and return types.

```
def reset(self, x: float, y: float, health: int, speed: float,  
          colour: Tuple[int, int, int], value: int, type_name: str) -> None:
```

As shown in the entities.py snippet above, the reset method strictly defines that the health must be an integer, that colour must be a Tuple containing three integers, and that the method will return None.

In addition to this, the flake8 linter was also integrated into the development pipeline. The linter parses the entire codebase before execution, enforcing the previously mentioned strict PEP-8 stylistic compliance. This will then flag any unused packages or imports, and identify any undeclared variables. This static analysis ensures that the engine remains highly readable and structurally robust at all times.

7.3 Automated Headless Regression Testing

The most complex software engineering process implemented in this project was the transition from manual playtesting to Automated Regression Testing. Using Python's built-in unittest framework, a suite of test cases was engineered to verify that the mathematical formulas driving the game logic, such as distance calculations and armour mitigation, all operated correctly.

However, testing a PyGame application presents a unique architectural challenge. By default, initialising PyGame forces the OS to allocate a physical window on the display

monitor. If an automated Continuous Integration (CI) pipeline or a terminal script attempts to run a PyGame test suite without a physical display attached, the application will instantly crash with a `pygame.error: video system not initialised`.

To solve this, a "Headless" execution environment was engineered within `test_logic.py`:

```
import os
import unittest

# --- HEADLESS MODE ---
# This tells PyGame to run without opening a graphical window.
# Crucial for professional automated testing!
os.environ["SDL_VIDEODRIVER"] = "dummy"

import pygame
pygame.init()
```

By intercepting the OS environment variables before PyGame initialises, the `SDL_VIDEODRIVER` is forced to utilise a "dummy" driver. This highly advanced technique deceives the underlying C-libraries into rendering the game's surfaces to a null video driver in system memory. This allows the unittest framework to execute the entire game loop, spawn entities, and calculate collision mathematics in a fraction of a second within the terminal, without ever drawing a pixel to a screen.

```
~/fyp$ python3 -m unittest tests/test_logic.py
<frozen importlib._bootstrap>:488: RuntimeWarning: Your system is avx2 capable but pygame was not built with support for
it. The performance of some of your blits could be adversely affected. Consider enabling compile time detection with en
vironment variables like PYGAME_DETECT_AVX2=1 if you are compiling without cross compilation.
pygame 2.5.2 (SDL 2.30.0, Python 3.12.3)
Hello from the pygame community. https://www.pygame.org/contribute.html
..Entering Gameplay State...
Gameplay logic thread started.
..
-----
Ran 4 tests in 0.019s
OK
```

Figure 2: Terminal output demonstrating the successful execution of the headless automated regression testing suite.

7.4 Mathematical Verification of Game Logic

With the headless environment established, the `TestGameStability` class was constructed to isolate and verify specific algorithmic behaviours.

A primary example of this is the verification of the polymorphism and mathematical mitigation of the "Armoured" enemy type. In the game rules, Armoured enemies possess a passive ability that reduces all incoming damage by 1. Rather than manually placing a tower within the GUI to visually confirm its functionality, a dedicated unit test was engineered to mathematically verify the interaction instantly:

```
# --- TEST 1: Enemy Damage & Armor Logic ---
def test_armoured_enemy_damage(self):
    """Tests that 'Armoured' enemies correctly reduce incoming damage by 1."""
    # Create an Armoured enemy with 8 Health
    armoured_enemy = Enemy(0, 0, health=8, speed=2, color=(0,0,0), value=20, type_name="Armoured")

    # Hit it for 3 damage. Because it is Armoured, it should only take 2 damage.
    armoured_enemy.take_damage(3)

    # 8 initial health - 2 actual damage = 6 remaining health
    self.assertEqual(armoured_enemy.health, 6, "Armoured enemy did not calculate damage reduction correctly.")
```

The test script explicitly instantiates the object, forces an interaction, and utilises `self.assertEqual()` to verify that the final internal state matches the expected mathematical outcome.

In addition, this framework was also used to verify the spatial mathematics within the Tower targeting system.

```
# --- TEST 2: Tower Targeting System ---
def test_tower_targeting_closest_enemy(self):
    """Tests that towers correctly identify enemies within their range."""
    # Place a tower at coordinates (50, 50)
    tower = BasicTower(50, 50)
    tower.attack_range = 100

    # Enemy 1 is at (50, 100) -> Distance is 50 pixels (IN RANGE)
    enemy_in_range = Enemy(50, 100, health=10, speed=1, color=(0,0,0), value=10, type_name="Basic")

    # Enemy 2 is at (250, 50) -> Distance is 200 pixels (OUT OF RANGE)
    enemy_out_of_range = Enemy(250, 50, health=10, speed=1, color=(0,0,0), value=10, type_name="Basic")

    # Pass both enemies to the tower's update loop
    tower.update([enemy_in_range, enemy_out_of_range])

    # The tower should have locked onto the enemy in range
    self.assertEqual(tower.target, enemy_in_range, "Tower failed to target the enemy within its range.")
```

In this test, the Euclidean distance algorithm is stress-tested. The `BasicTower` is provided with an array containing two enemies, one inside its geometric radius, and one outside. By asserting that `tower.target` resolves exclusively to `enemy_in_range`, the test mathematically guarantees that the tower's targeting algorithm is completely accurate. Integrating this regression suite ensures that future modifications to the codebase cannot silently break core engine mechanics.

Chapter 8: Critical Evaluation and Performance Profiling

8.1 Introduction to Empirical Validation

While the theoretical foundations and architectural patterns established in the preceding chapters can strongly suggest a highly optimised game engine, professional software engineering mandates empirical validation. However, a system cannot be considered robust until it has been rigorously stress-tested under extreme simulated conditions.

The purpose of this chapter is to critically evaluate the practical execution of the engine. By isolating specific systems, namely the rendering pipeline, the Object Pool memory manager, and the A* pathfinding matrix, this section provides a high-level quantitative and qualitative analysis of the engine's performance. The objective is to determine exactly how the Python architecture handles massive entity scaling, to identify the engine's upper computational limits, and to empirically justify the complex design decisions made during development.

8.2 Rendering Performance and Entity Scaling

A primary metric for evaluating any real-time interactive media is the stability of its frame rate. The performance target that was set as the baseline for this project was a consistent 60 Frames Per Second (FPS), which requires a strict execution budget of exactly 16.6 ms per frame. If the combined tasks of event polling, game state updating, and buffer drawing exceed this 16.6 ms threshold, the application drops frames, which would lead to visual latency.

To evaluate the scalability of the engine, the system was subjected to artificial stress tests that incremented the active enemy count (N).

Baseline Load (N = 10): Under standard early-wave conditions, the engine was easily able to maintain a locked 60 FPS. The CPU remained largely idle, and the PyGame blitting operations (rendering the 20 x 20 pixel enemy surfaces) consumed less than 2 ms of the frame budget.

Heavy Load (N = 100): As the entity count scaled to 100 simultaneous actors, the limits of Python's single-core execution became apparent in the Logic Thread. However, the true success of the concurrent architecture was validated here. While calculating the movement vectors and distance checks for 100 enemies took around 12 ms to 15 ms per tick, the Main UI Thread remained completely unaffected throughout the process. Due to this the presentation layer continued to render at a flawless 60 FPS, proving that the engine successfully decoupled visual rendering from mathematical simulation.

Extreme Stress Test (N = 200+): With 200 active enemies, the Logic Thread formally surpassed the 16.6 ms threshold, occasionally performing an update loop in 25 ms. In a traditional, single-threaded procedural engine, this would have caused severe UI stuttering, reducing the frame rate to roughly 40 frames per second and making button clicks useless. Nevertheless, the UI thread maintained 60 FPS due to the Monitor Object pattern and asynchronous queue architecture. The only noticeable deterioration was that the enemies updated their spatial locations a little less regularly, which caused a tiny "ghosting" effect instead of a freeze of the entire application. This

essentially supports the choice to use the threading module despite the Global Interpreter Lock's (GIL) drawbacks.

8.3 Critical Evaluation of the Object Pool

One of the most complex architectural decisions in this project was the implementation of the EnemyPool to bypass Python's native memory management. To critically evaluate this decision, it is necessary to compare the theoretical memory profile of a naive dynamic allocation approach against the observed results of the implemented static Object Pool.

8.3.1 The "Sawtooth" Profile of Dynamic Allocation

If the engine had relied on standard Python instantiation, such as `new_enemy = Enemy()`, the memory profile during a high-density wave would resemble a "sawtooth" graph. As enemies spawn, RAM consumption linearly increases. When a tower defeats an enemy, the object's reference count drops to zero. However, the memory is not instantly freed. Instead, it becomes fragmented until Python's generational cyclic garbage collector initiates a sweep.

During stress testing of early prototypes, this automated sweep caused extreme execution spikes. The Garbage Collector operates as a "stop-the-world" process, it forcefully halted both the Logic and UI threads for upwards of 40 ms to reclaim memory sectors. These GC spikes caused severe, unpredictable frame drops that completely undermined the stability of the simulation.

8.3.2 The "Flatline" Profile of Static Allocation

By implementing the Object Pool pattern, the engine forces the memory profile into a strict "flatline." During the initialisation of the `GameplayState`, 100 Enemy objects are pre-allocated, causing an initial, acceptable spike in memory and a minor increase in loading screen duration.

During active gameplay, the memory footprint remains entirely static. When an enemy is defeated, calling `self.enemies.remove(enemy)` removes it from the PyGame rendering group, but the `EnemyPool` array maintains a permanent reference to the object. Because `sys.getrefcount()` never evaluates to zero, the Garbage Collector ignores the active gameplay loop entirely. Frame times dropped from highly fluctuating, unpredictable millisecond counts to a rock-solid, predictable execution time. This empirical evidence proves that while Python is heavily automated, manually overriding its memory allocation paradigms is an absolute necessity for high-performance interactive development.

8.4 Empirical Analysis of A* Pathfinding Complexity

While the A* algorithm is the industry standard for spatial navigation, its execution within a Python environment introduces the heaviest computational bottleneck in the engine. It is essential to critically evaluate the algorithmic time complexity of the pathfinding system in actual practice.

8.4.1 Baseline Algorithmic Complexity

The mathematical time complexity of A* is highly dependent on the employed heuristic. In the worst-case scenario where the heuristic completely fails to prune the search space, the algorithm degrades into Dijkstra's Algorithm, which operates at $O(E + V \log V)$.

However, because the engine restricts movement to a 2D Cartesian grid and utilises the Manhattan Distance calculation ($h(n) = |x_1 - x_2| + |y_1 - y_2|$), the search space is aggressively pruned. In real-world profiling, the algorithm seldom considers more than 15% of all grid nodes before successfully charting a route to the goal when an enemy appears at the beginning of the path. The base construction of a single path is kept under 2 ms by integrating Python's `heapq` module, which guarantees that fetching the lowest `f_cost` node stays an $O(1)$ constant time operation.

8.4.2 The $O(K \cdot (V \log V))$ Bottleneck

During gameplay, the existing pathfinding architecture's main issue becomes apparent. Due to the "Mazing" mechanic, every active enemy on the map must immediately recalculate its path when a player creates a new tower in order to avoid walking through the new obstacle, according to the "Mazing" principle.

If there are K active enemies on the screen, the Logic Thread must execute the A* algorithm K consecutive times. Therefore, the dynamic rerouting complexity scales to $O(K \cdot (V \log V))$. During profiling, if a player placed a tower while 50 enemies were active, the Logic Thread was forced to execute 50 distinct A* evaluations sequentially.

This resulted in a massive computational spike. The Logic Thread would "hang" for up to 150 ms while processing the pathfinding queue, yet the UI thread remained flawlessly responsive, enabling the player to keep moving their mouse. For a brief moment, the enemies would remain still as they determined their new paths, and then they would move again.

8.4.3 Evaluation of Mitigation Strategies

This profiling exposes a fundamental limitation of the current architecture. While the concurrent thread separation prevents the application from crashing or the OS from flagging it as "Unresponsive," the CPU overhead of mass A* recalculation is still too heavy for Python to handle seamlessly.

The engine would have to switch from individual agent-based pathfinding to a Flow Field (Vector Field) pathfinding algorithm in order to address this in a professional setting. In a Flow Field, the A* algorithm is only executed once per grid update, generating a global directional map that all K enemies query simultaneously in $O(1)$ time. Identifying this particular algorithmic bottleneck via empirical profiling shows a thorough, critical comprehension of engine architecture and directly influences the specifications for future technical developments.

Chapter 9: Professional Issues and Conclusion

9.1 Introduction to Professional Context

The development of a software engine goes beyond just the implementation of mathematical algorithms and architectural design patterns. To adhere to the professional standards that are expected within the discipline of Computer Science, the artifact must be evaluated in a broader societal, legal, and ethical context. Software engineers possess a professional obligation to ensure that the systems they architect are legally compliant, accessible to diverse user bases, and maintainable for future development cycles. This section critically evaluates the project against these professional metrics.

9.2 Legal and Intellectual Property (IP) Considerations

A primary legal consideration in modern software engineering is the management of Intellectual Property (IP) and adherence to open-source licensing. This project fundamentally relies on the Python standard library and the PyGame module. PyGame is distributed under the GNU Lesser General Public License (LGPL).

From a professional perspective, using an LGPL framework allows the engine to be distributed commercially or academically without requiring the project's proprietary source code to be made publicly available, as long as the underlying PyGame library is dynamically linked rather than altered. By isolating the engine's core logic entirely from the underlying PyGame wrapper, specifically by containing all rendering calls within discrete `draw()` methods and maintaining state strictly via mathematics, the engine ensures strict compliance with these licensing boundaries.

Furthermore, the production of video games often presents significant intellectual property challenges with graphical and aural content. To completely mitigate the risk of copyright infringement, this project deliberately avoids using external, third-party sprite sheets or audio recordings downloaded from the internet. Instead, all visual representations were created algorithmically at runtime using geometric rendering techniques like `pygame.draw.circle` and `pygame.draw.rect`. This guarantees that the entire software and visual output are 100% original intellectual property and may be published in academic journals without facing legal obstacles.

9.3 Ethical Implications and Hardware Accessibility

While video games are mainly entertainment media, the architectural decisions made during their development carry clear ethical and social implications, primarily in regard to accessibility.

Within the modern gaming industry, heavily unoptimised engines fundamentally restrict access exclusively to users possessing high-end, expensive computer hardware. Therefore, this creates a socio-economic barrier to entry. From an ethical engineering perspective, the aggressive optimisation strategies implemented within this project, especially the multi-threaded circumvention of the Global Interpreter Lock (GIL) and the use of the Object Pool pattern to eliminate Garbage Collection (GC) spikes, serve as a democratising factor. By guaranteeing a flat memory footprint and offloading A* pathfinding to a background thread, the engine ensures that the simulation is able to run smoothly at 60 Frames Per Second (FPS) on severely hardware-constrained machines, promoting broader digital accessibility.

In addition, cognitive and visual accessibility was a driving factor behind the implementation of the `Data-Driven settings.py` architecture. Because all colours, such as `COLOUR_ENEMY_FAST`, and `COLOUR_TOWER_SPLASH`, are all defined as global constant tuples rather than being hardcoded into the rendering logic, a future developer could potentially implement a "Colour-blind Mode" by simply overriding the settings file at runtime, ensuring the software adheres to inclusive design principles.

9.4 Professional Standards and Maintainability

Professional engineers are required by the British Computer Society (BCS) to produce software that is strong, dependable, and maintainable. Delivering fragile, undocumented 'spaghetti code' is considered a breach of professional standards.

This project rigorously adhered to these standards through the implementation of the software engineering processes detailed in Chapter 6. The enforcement of PEP-8 styling via the flake8 linter, combined with strict PEP 484 Type Hinting, guarantees that the

codebase can be handed over to a separate engineering team without a loss of structural understanding. The addition of a headless Automated Regression Testing suite, which offers mathematical evidence that the product operates as promised and is thus shielded against future regression errors, further demonstrates a professional commitment to Quality Assurance (QA).

9.5 Project Evaluation and Conclusion

The primary objective of this project was to create a modular, 2D Tower Defence engine from the ground up, with a focus on keeping game logic and graphics separate to ensure Python performance stability. This objective has been achieved.

The initial procedural prototype was successfully torn down and rebuilt using robust Object-Oriented Design Patterns. The implementation of the State Pattern allowed for the elimination of flimsy monolithic loops, while the Factory Pattern established a highly scalable and data-driven entity instantiation system. The most significant technical hurdle was Python's Global Interpreter Lock (GIL), which was successfully navigated. By engineering a thread-safe Event Queue and utilising Monitor Objects (`threading.Lock`), the engine achieved true concurrent execution, preventing the PyGame UI thread from freezing during computationally intensive A* heuristic graph traversals.

Ultimately, this project served as a rigorous investigation into the "black box" of game engine architecture. By manually engineering the systems that heavyweight engines like Unity3D typically abstract away, the project demonstrated a mastery over advanced memory management, multi-threading, and algorithmic efficiency.

9.5.1 Project Self-Evaluation

When considering the overall development process, the project's main goal was to create a high-performance, concurrent simulation in Python, which was accomplished with great success. If I were to evaluate what went right, the early commitment to strict Object-Oriented design patterns and the implementation of the Object Pool entirely saved the codebase from becoming unmaintainable as complexity scaled. However, I made critical miscalculations early on, I heavily underestimated PyGame's rendering limits and the sheer difficulty of bypassing Python's Global Interpreter Lock (GIL). I quickly learned that naive multi-threading creates more problems than it solves and safely synchronising the Logic and UI threads using Mutex locks required a substantial, unplanned architectural redesign. Ultimately, this project taught me that professional software engineering relies far more on iterative profiling and architectural foresight than simply writing code. In order to address the remaining A* computational bottlenecks, this engine's next natural step is to incorporate Quadtree spatial

partitioning and Flow Field algorithms. This will eventually open the door to translating these ideas into a natively built language like C++.

9.6 Future Work

Although the engine is now stable and feature-rich in relation to its original academic scope, the architecture's high degree of decoupling offers a solid basis for upcoming technical developments:

- **Spatial Partitioning (Quadrees):** Currently, the Tower classes scan for targets by iterating over every living enemy and calculating Euclidean distances (an $O(N)$ operation per tower). As entity counts scale over 1,000, this will inevitably become a bottleneck over time. Future iterations will implement a Quadtree spatial partitioning algorithm to subdivide the 2D plane. This will reduce the collision detection complexity from $O(N^2)$ to $O(N \log N)$.
- **State Serialisation (Save Systems):** To transition the engine into a commercial product, a serialisation system must be implemented. The current data-driven architecture is highly conducive to this, future work will involve dumping the GameState entity lists and PlayerBank integers into a .json file, allowing users to persist their simulation state between sessions.
- **Deployment with PyInstaller:** Using PyInstaller to compile the Python scripts into a standalone .exe executable is the last logical step. This will guarantee that users can receive the product without having to install extra dependencies using pip or set up a Python runtime environment.

Bibliography

- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009). Introduction to Algorithms. 3rd ed. Cambridge, MA: MIT Press.
- Dijkstra, E.W. (1959). 'A note on two problems in connexion with graphs'. Numerische Mathematik, 1(1), pp.269-271.
- Donovan, A.A. and Kernighan, B.W. (2015). The Go Programming Language. Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.
- Gregory, J. (2018). Game Engine Architecture. 3rd ed. Boca Raton, FL: CRC Press.
- Hart, P.E., Nilsson, N.J. and Raphael, B. (1968). 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths'. IEEE Transactions on Systems Science and Cybernetics, 4(2), pp.100-107.
- Hoare, C.A.R. (1978). 'Communicating sequential processes'. Communications of the ACM, 21(8), pp.666-677.
- Mikhajlov, I. and Sekerinski, E. (1998). 'A study of the fragile base class problem'. In European Conference on Object-Oriented Programming (pp. 355-369). Springer, Berlin, Heidelberg.
- Nystrom, R. (2014). Game Programming Patterns. Genever Benning.
- Python Software Foundation (2023). Python 3.12.0 Documentation. [online] Available at: <https://docs.python.org/3/> [Accessed 10 Apr. 2026].
- Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach. 4th ed. Harlow: Pearson.
- Shinnars, P. (2023). PyGame Documentation. [online] Available at: <https://www.pygame.org/docs/> [Accessed 10 Apr. 2026].
- Williams, A. (2012). C++ Concurrency in Action: Practical Multithreading. Shelter Island, NY: Manning Publications.
- Zechner, M. (2011). Beginning Android Games. Apress.

Appendices

Appendix A: Advanced Spatial Navigation (A* Algorithm)

```
import heapq
from typing import List, Tuple, Optional, Set

class GridNode:
    """Represents a single cell within the A* Pathfinding grid."""
    def __init__(self, x: int, y: int) -> None:
        self.x: int = x
        self.y: int = y
        self.walkable: bool = True
        self.g_cost: int = 0 # Cost from start node
        self.h_cost: int = 0 # Heuristic
        self.parent: Optional['GridNode'] = None

    @property
    def f_cost(self) -> int:
        """Calculates the total heuristic cost of the node."""
        return self.g_cost + self.h_cost

    def __lt__(self, other: 'GridNode') -> bool:
        """Enables sorting within the heapq module by f_cost."""
        return self.f_cost < other.f_cost

class Pathfinder:
    """Manages the A* Grid and executes shortest-path calculations."""
    def __init__(self, width: int, height: int, cell_size: int = 40) -> None:
        self.cell_size: int = cell_size
        self.cols: int = int(width // cell_size)
        self.rows: int = int(height // cell_size)
        self.grid: List[List[GridNode]] = self._create_grid()

    def _create_grid(self) -> List[List[GridNode]]:
        """Generates the 2D array of GridNodes."""
        grid = []
        for x in range(self.cols):
            column = []
            for y in range(self.rows):
                column.append(GridNode(x, y))
            grid.append(column)
        return grid

    def _reset_grid(self) -> None:
        """Clears the A* calculations from previous operations."""
        for x in range(self.cols):
            for y in range(self.rows):
                self.grid[x][y].g_cost = 0
                self.grid[x][y].h_cost = 0
                self.grid[x][y].parent = None

    def get_node(self, world_x: float, world_y: float) -> Optional[GridNode]:
```

```

    """Converts raw pixel coordinates to grid coordinates."""
    grid_x = int(world_x // self.cell_size)
    grid_y = int(world_y // self.cell_size)
    if 0 <= grid_x < self.cols and 0 <= grid_y < self.rows:
        return self.grid[grid_x][grid_y]
    return None

def get_neighbours(self, node: GridNode) -> List[GridNode]:
    """Retrieves adjacent nodes for path exploration."""
    neighbours = []
    directions = [(0, -1), (0, 1), (-1, 0), (1, 0)]
    for dx, dy in directions:
        check_x = node.x + dx
        check_y = node.y + dy
        if 0 <= check_x < self.cols and 0 <= check_y < self.rows:
            neighbours.append(self.grid[check_x][check_y])
    return neighbours

def find_path(
    self,
    start_world_pos: Tuple[float, float],
    end_world_pos: Tuple[float, float]
) -> List[Tuple[float, float]]:
    """
    Executes the A* Algorithm to find the optimal route.

    Args:
        start_world_pos (Tuple[float, float]): Starting X/Y pixels.
        end_world_pos (Tuple[float, float]): Target X/Y pixels.

    Returns:
        List[Tuple[float, float]]: A list of pixel coordinates forming the path.
    """
    self._reset_grid()

    start_node = self.get_node(start_world_pos[0], start_world_pos[1])
    end_node = self.get_node(end_world_pos[0], end_world_pos[1])

    if not start_node or not end_node:
        return []

    open_set: List[Tuple[int, GridNode]] = []
    closed_set: Set[GridNode] = set()

    heapq.heappush(open_set, (0, start_node))

    while open_set:
        current_node = heapq.heappop(open_set)[1]
        closed_set.add(current_node)

        if current_node == end_node:
            return self._retrace_path(start_node, end_node)

        for neighbour in self.get_neighbours(current_node):
            if not neighbour.walkable or neighbour in closed_set:

```



```

        continue
    move_cost = current_node.g_cost + 1
    if move_cost < neighbour.g_cost or neighbour.g_cost == 0:
        neighbour.g_cost = move_cost
        neighbour.h_cost = abs(neighbour.x - end_node.x) + abs(neighbour.y -
end_node.y)

        neighbour.parent = current_node
        if not any(n == neighbour for _, n in open_set):
            heapq.heappush(open_set, (neighbour.f_cost, neighbour))

    return []

def _retrace_path(self, start_node: GridNode, end_node: GridNode) -> List[Tuple[float,
float]]:
    """Traces the parents backward from the destination to map the route."""
    path = []
    current_node = end_node
    while current_node != start_node:
        world_x = (current_node.x * self.cell_size) + (self.cell_size // 2)
        world_y = (current_node.y * self.cell_size) + (self.cell_size // 2)
        path.append((world_x, world_y))
        if current_node.parent:
            current_node = current_node.parent
    path.reverse()
    return path

```

Appendix B: Concurrent State Management

```

import pygame
import threading
import queue
import time

# --- FIX: Import the settings module itself ---
import settings
from factories import enemy_pool, TowerFactory
from ui import Button, Subject
from pathfinding import Pathfinder

class PlayerBank(Subject):
    def __init__(self, starting_currency):
        super().__init__()
        self._currency = starting_currency

    @property
    def currency(self):
        return self._currency

    @currency.setter
    def currency(self, value):
        self._currency = value
        self.notify("currency_changed", self._currency)

```

```

# --- 4. State Pattern Implementation ---

class GameState:
    """Base class for all game states."""
    def __init__(self, state_manager):
        self.state_manager = state_manager

    def handle_events(self, events):
        pass

    def update(self):
        pass

    def draw(self, screen):
        pass

    def on_enter(self):
        pass

    def on_exit(self):
        pass

class MainMenuState(GameState):
    """The state for the main menu screen."""
    def __init__(self, state_manager):
        super().__init__(state_manager)
        self.buttons = []
        self.font_title = pygame.font.Font(None, 100)
        self._setup_ui()

    def _setup_ui(self):
        def start_game_action():
            print("--- START GAME ---")
            self.state_manager.set_state("gameplay")

        start_button = Button(
            x=settings.SCREEN_WIDTH // 2 - 150, y=settings.SCREEN_HEIGHT // 2,
            width=300, height=70,
            text="START GAME",
            colour=settings.colour_BLUE,
            hover_colour=settings.colour_RED,
            action=start_game_action
        )
        self.buttons.append(start_button)

    def handle_events(self, events):
        for event in events:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    self.state_manager.running = False
            for button in self.buttons:
                action = button.handle_event(event)
                if action:

```

```

        action()

    def draw(self, screen):
        screen.fill(settings.colour_BLACK)

        title_surf = self.font_title.render("2D Tower Defence", True,
settings.colour_WHITE)
        title_rect = title_surf.get_rect(center=(settings.SCREEN_WIDTH // 2,
settings.SCREEN_HEIGHT // 2 - 150))
        screen.blit(title_surf, title_rect)

        for button in self.buttons:
            button.draw(screen)

class GameState(GameState):
    """The main gameplay state, containing all game logic."""
    def __init__(self, state_manager):
        super().__init__(state_manager)

        self.data_lock = threading.Lock()
        self.event_queue = queue.Queue()
        self.logic_thread = None
        self.logic_running = False

        self.towers = pygame.sprite.Group()
        self.enemies = pygame.sprite.Group()

        self.buttons = []
        self.font_small = pygame.font.Font(None, 30)
        self.font_medium = pygame.font.Font(None, 40)

        self.bank = PlayerBank(settings.PLAYER_STARTING_CURRENCY)
        self.player_health = settings.PLAYER_STARTING_HEALTH

        self.selected_tower_type = None
        self.selected_placed_tower = None
        self.build_cursor_colour = None
        self.build_buttons = {}
        self.start_wave_button = None
        self.upgrade_button = None

        self.enemy_count = 0
        self.spawn_timer = 0
        self.spawn_rate = settings.FPS * 0.5

        self.wave_index = 0
        self.enemy_index_in_wave = 0
        self.current_wave_list = []
        self.wave_running = False
        self.game_over = False
        self.game_won = False

    def on_enter(self):
        print("Entering Gameplay State...")

```

```

self._setup_ui()

self.pathfinder = Pathfinder(settings.GAME_AREA_WIDTH, settings.SCREEN_HEIGHT,
cell_size=40)
self.start_pos = (20, settings.SCREEN_HEIGHT // 2)
self.end_pos = (settings.GAME_AREA_WIDTH - 20, settings.SCREEN_HEIGHT // 2)

settings.PATH_POINTS = self.pathfinder.find_path(self.start_pos, self.end_pos)

self.logic_running = True
self.logic_thread = threading.Thread(target=self.run_game_logic, daemon=True)
self.logic_thread.start()

def on_exit(self):
    print("Exiting Gameplay State...")
    self.logic_running = False
    self.event_queue.put("QUIT")
    if self.logic_thread:
        self.logic_thread.join()
    print("Gameplay logic thread stopped.")

def _setup_ui(self):

    def start_wave_action():
        if not self.wave_running and not self.game_won:
            self.event_queue.put("START_WAVE")

    self.start_wave_button = Button(
        x=settings.GAME_AREA_WIDTH + 40, y=140,
        width=220, height=50,
        text="START WAVE",
        colour=settings.colour_GREEN,
        hover_colour=settings.colour_RED,
        action=start_wave_action
    )
    self.buttons.append(self.start_wave_button)

    def make_select_action(tower_type, colour):
        def on_select():
            print(f"Selected {tower_type} Tower")
            self.selected_tower_type = tower_type
            self.selected_placed_tower = None
            self.build_cursor_colour = colour
        return on_select

    button_data = ["Basic", "Splash", "Slow", "Sniper", "Rapid"]
    start_y = 280
    y_offset = 70
    for i, tower_type in enumerate(button_data):
        stats = settings.TOWER_STATS[tower_type]
        cost = stats["cost"]
        colour = stats["colour"]

        button = Button(
            x=settings.GAME_AREA_WIDTH + 40, y=start_y + (i * y_offset),

```

```

        width=220, height=60,
        text=f"{tower_type} ({cost}g)",
        colour=colour,
        hover_colour=settings.colour_GREEN,
        action=make_select_action(tower_type, colour),
        cost=cost
    )
    self.buttons.append(button)
    self.build_buttons[tower_type] = button
    self.bank.add_observer(button)

# --- UPGRADE BUTTON ---
def upgrade_action():
    if self.selected_placed_tower:
        self.event_queue.put(("UPGRADE_TOWER", self.selected_placed_tower))

self.upgrade_button = Button(
    x=settings.GAME_AREA_WIDTH + 40, y=settings.SCREEN_HEIGHT - 100,
    width=220, height=60, text="SELECT TOWER",
    colour=settings.colour_BLUE, hover_colour=settings.colour_GREEN,
    action=upgrade_action
)
self.buttons.append(self.upgrade_button)
self.bank.add_observer(self.upgrade_button)

def handle_events(self, events):
    if self.game_over or self.game_won:
        return

    for event in events:
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_ESCAPE:
                self.state_manager.running = False

        for button in self.buttons:
            action = button.handle_event(event)
            if action:
                action()

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            # 1. Selecting an existing placed tower
            if not self.selected_tower_type and event.pos[0] <
settings.GAME_AREA_WIDTH:
                clicked_tower = None
                for tower in self.towers:
                    if tower.rect.collidepoint(event.pos):
                        clicked_tower = tower
                        break
                self.selected_placed_tower = clicked_tower

            # 2. Building a new tower
            if self.selected_tower_type and event.pos[0] < settings.GAME_AREA_WIDTH:
                build_pos = event.pos
                self.event_queue.put(

```

```

        ("BUILD_TOWER", self.selected_tower_type, build_pos)
    )
    self.selected_tower_type = None
    self.selected_placed_tower = None
    self.build_cursor_colour = None

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
        if self.selected_tower_type:
            print("Build mode cancelled.")
            self.selected_tower_type = None
            self.build_cursor_colour = None
        if self.selected_placed_tower:
            self.selected_placed_tower = None

def draw_path(self, screen):
    if len(settings.PATH_POINTS) > 1:
        pygame.draw.lines(screen, settings.colour_PATH, False, settings.PATH_POINTS, 5)

def draw(self, screen):
    # --- Handle State Transitions ---
    if self.game_over:
        self.state_manager.set_state("game_over")
        return
    if self.game_won:
        self.state_manager.set_state("victory")
        return

    screen.fill(settings.colour_BLACK)
    self.draw_path(screen)

    with self.data_lock:
        for enemy in self.enemies:
            enemy.draw(screen)
        for tower in self.towers:
            tower.draw(screen)
        self.enemy_count = len(self.enemies)

    ui_panel_rect = pygame.Rect(settings.GAME_AREA_WIDTH, 0, settings.UI_PANEL_WIDTH,
settings.SCREEN_HEIGHT)
    pygame.draw.rect(screen, settings.colour_DARK_GREY, ui_panel_rect)

    with self.data_lock:
        self.start_wave_button.disabled = self.wave_running
        if self.game_won:
            self.start_wave_button.text = "YOU WIN!"
            self.start_wave_button.disabled = True
        elif self.wave_running:
            self.start_wave_button.text = "Wave in Progress"
        else:
            self.start_wave_button.text = f"START WAVE {self.wave_index + 1}"

        currency_surf = self.font_medium.render(f"Gold: {self.bank.currency}", True,
settings.colour_CURRENCY)
        screen.blit(currency_surf, (settings.GAME_AREA_WIDTH + 30, 50))

```

```

        health_surf = self.font_medium.render(f"Health: {self.player_health}", True,
settings.colour_HEALTH_RED)
        screen.blit(health_surf, (settings.GAME_AREA_WIDTH + 30, 85))

        text_surf = self.font_small.render(f"Enemies: {self.enemy_count}", True,
settings.colour_WHITE)
        screen.blit(text_surf, (settings.GAME_AREA_WIDTH + 30, 220))

# --- Dynamic Upgrade UI Updates ---
if self.selected_placed_tower:
    # Draw a white ring around the selected tower showing its current range
    pygame.draw.circle(screen, settings.colour_WHITE,
self.selected_placed_tower.rect.center, self.selected_placed_tower.attack_range, 1)

    # Dynamically update the upgrade button based on the selected tower
    self.upgrade_button.cost = self.selected_placed_tower.upgrade_cost
    self.upgrade_button.text = f"UPGRADE: Lvl {self.selected_placed_tower.level+1}
({self.upgrade_button.cost}g)"
    self.upgrade_button.disabled = self.bank.currency < self.upgrade_button.cost
else:
    self.upgrade_button.text = "SELECT TOWER"
    self.upgrade_button.disabled = True

for button in self.buttons:
    button.draw(screen)

if self.selected_tower_type:
    mouse_pos = pygame.mouse.get_pos()
    if mouse_pos[0] < settings.GAME_AREA_WIDTH:
        radius = settings.TOWER_STATS[self.selected_tower_type]["range"]

        s = pygame.Surface((radius * 2, radius * 2), pygame.SRCALPHA)
        s.fill((0,0,0,0))
        pygame.draw.circle(s, self.build_cursor_colour + (100,), (radius, radius),
radius)

        pygame.draw.rect(s, self.build_cursor_colour + (200,), (radius-20, radius-
20, 40, 40), border_radius=5)

        screen.blit(s, (mouse_pos[0] - radius, mouse_pos[1] - radius))

# --- Logic Thread Methods ---

def _start_wave_logic(self):
    if self.wave_running or self.game_won:
        return

    if self.wave_index >= len(settings.WAVES):
        print("All waves completed!")
        with self.data_lock:
            self.game_won = True
        return

    self.wave_running = True

```

```

self.current_wave_list = settings.WAVES[self.wave_index]
self.wave_index += 1
self.enemy_index_in_wave = 0
self.spawn_timer = 0
print(f"--- Starting Wave {self.wave_index} ---")

def _spawn_next_enemy_logic(self):
    if self.enemy_index_in_wave < len(self.current_wave_list):
        enemy_type = self.current_wave_list[self.enemy_index_in_wave]
        self.enemy_index_in_wave += 1

        new_enemy = enemy_pool.get_enemy(enemy_type)
        with self.data_lock:
            if not self.enemies.has(new_enemy):
                self.enemies.add(new_enemy)

def _build_tower_action_logic(self, tower_type, position):
    stats = settings.TOWER_STATS.get(tower_type)
    if stats is None:
        return

    cost = stats["cost"]
    with self.data_lock:
        if self.bank.currency < cost:
            print(f"Not enough currency to build {tower_type}.")
            return

        node = self.pathfinder.get_node(position[0], position[1])
        if not node or not node.walkable:
            print("Cannot build here.")
            return

        node.walkable = False
        new_path = self.pathfinder.find_path(self.start_pos, self.end_pos)

        if not new_path:
            print("Build failed: This tower would trap the enemies!")
            node.walkable = True
            return

        self.bank.currency -= cost
        settings.PATH_POINTS = new_path

        for enemy in self.enemies:
            enemy_path = self.pathfinder.find_path(enemy.rect.center, self.end_pos)
            if enemy_path:
                enemy.path = enemy_path
                enemy.current_path_index = 0
                enemy.target_point = enemy.path[0]

    grid_x = (node.x * self.pathfinder.cell_size)
    grid_y = (node.y * self.pathfinder.cell_size)

```



```

        try:
            new_tower = TowerFactory.create_tower(tower_type, grid_x, grid_y)
            self.towers.add(new_tower)
            print("Build successful. Enemies rerouted!")
        except Exception as e:
            print(f"Build failed: {e}")

def update(self):
    if self.game_over or self.game_won:
        return

    if self.wave_running:
        self.spawn_timer += 1
        if self.spawn_timer >= self.spawn_rate:
            self._spawn_next_enemy_logic()
            self.spawn_timer = 0

    with self.data_lock:
        self.enemies.update()
        self.towers.update(self.enemies)

    for enemy in list(self.enemies):
        if not enemy.alive:
            if enemy.current_path_index >= len(enemy.path):
                self.player_health -= 1
                self.enemies.remove(enemy)
                print(f"Enemy reached the end! Health remaining:
{self.player_health}")
            if self.player_health <= 0:
                print("GAME OVER - Player health is zero")
                self.game_over = True
                self.logic_running = False
            else:
                self.bank.currency += enemy.value
                self.enemies.remove(enemy)

        if self.wave_running and self.enemy_index_in_wave >=
len(self.current_wave_list) and len(self.enemies) == 0:
            self.wave_running = False
            print("Wave complete.")

        # Check if that was the final wave
        if self.wave_index >= len(settings.WAVES):
            print("VICTORY SECURED!")
            self.game_won = True
            self.logic_running = False

def run_game_logic(self):
    """The main loop for the logic thread (LOGIC THREAD)."""
    print("Gameplay logic thread started.")
    last_update_time = time.perf_counter()

    while self.logic_running:

```

```

        try:
            event = self.event_queue.get_nowait()
            if event == "QUIT":
                self.logic_running = False
            elif event == "START_WAVE":
                self._start_wave_logic()

            elif isinstance(event, tuple):
                if event[0] == "BUILD_TOWER":
                    self._build_tower_action_logic(event[1], event[2])
                elif event[0] == "UPGRADE_TOWER":
                    tower_to_upgrade = event[1]
                    with self.data_lock:
                        if self.bank.currency >= tower_to_upgrade.upgrade_cost:
                            self.bank.currency -= tower_to_upgrade.upgrade_cost
                            tower_to_upgrade.upgrade()

        except queue.Empty:
            pass

        current_time = time.perf_counter()
        if (current_time - last_update_time) > (1.0 / settings.FPS):
            self.update()
            last_update_time = current_time

class GameOverState(GameState):
    """The screen shown when health reaches zero."""
    def __init__(self, state_manager):
        super().__init__(state_manager)
        self.font_large = pygame.font.Font(None, 120)

    def restart_action():
        self.state_manager.set_state("gameplay")
    def menu_action():
        self.state_manager.set_state("main_menu")

    self.buttons = [
        Button(settings.SCREEN_WIDTH//2 - 150, settings.SCREEN_HEIGHT//2 + 50, 300, 60,
"RESTART", settings.colour_GREEN, settings.colour_BLUE, restart_action),
        Button(settings.SCREEN_WIDTH//2 - 150, settings.SCREEN_HEIGHT//2 + 130, 300,
60, "MAIN MENU", settings.colour_DARK_GREY, settings.colour_RED, menu_action)
    ]

    def handle_events(self, events):
        for event in events:
            for button in self.buttons:
                action = button.handle_event(event)
                if action: action()

    def draw(self, screen):
        screen.fill(settings.colour_BLACK)
        text_surf = self.font_large.render("GAME OVER", True, settings.colour_RED)
        screen.blit(text_surf, text_surf.get_rect(center=(settings.SCREEN_WIDTH // 2,
settings.SCREEN_HEIGHT // 2 - 100)))

```

```

        for button in self.buttons: button.draw(screen)

class VictoryState(GameState):
    """The screen shown when all waves are defeated."""
    def __init__(self, state_manager):
        super().__init__(state_manager)
        self.font_large = pygame.font.Font(None, 120)

        def menu_action():
            self.state_manager.set_state("main_menu")

        self.buttons = [
            Button(settings.SCREEN_WIDTH//2 - 150, settings.SCREEN_HEIGHT//2 + 50, 300, 60,
"MAIN MENU", settings.colour_BLUE, settings.colour_GREEN, menu_action)
        ]

    def handle_events(self, events):
        for event in events:
            for button in self.buttons:
                action = button.handle_event(event)
                if action: action()

    def draw(self, screen):
        screen.fill(settings.colour_BLACK)
        text_surf = self.font_large.render("VICTORY!", True, settings.colour_CURRENCY)
        screen.blit(text_surf, text_surf.get_rect(center=(settings.SCREEN_WIDTH // 2,
settings.SCREEN_HEIGHT // 2 - 100)))
        for button in self.buttons: button.draw(screen)

```

Appendix C: Memory Management (The Object Pool)

```

import settings
from entities import (
    Enemy, BasicTower, SplashTower, SlowTower, SniperTower, RapidTower
)

# --- 2. Factory Pattern Implementation ---
class EnemyPool:
    """Implements the Object Pool Pattern for memory efficiency."""
    def __init__(self, initial_size=100):
        self.pool = []
        # Pre-allocate 100 inactive enemies into memory at startup
        start_pos = settings.PATH_POINTS[0]
        for _ in range(initial_size):
            enemy = Enemy(start_pos[0], start_pos[1], 1, 1, (0,0,0), 0, "None")
            enemy.alive = False
            self.pool.append(enemy)

    def get_enemy(self, type_name):
        """Retrieves an inactive enemy from the pool and resets it."""
        start_pos = settings.PATH_POINTS[0]

        # Define base stats
        stats = {
            "Basic": (3, 2, settings.colour_ENEMY_BASIC, settings.ENEMY_REWARDS["Basic"]),

```

```

        "Fast": (2, 4, settings.colour_ENEMY_FAST, settings.ENEMY_REWARDS["Fast"]),
        "Tank": (15, 1, settings.colour_ENEMY_TANK, settings.ENEMY_REWARDS["Tank"]),
        "Armoured": (8, 2, settings.colour_ENEMY_Armoured,
settings.ENEMY_REWARDS["Armoured"]),
        "Boss": (150, 1, settings.colour_ENEMY_BOSS, settings.ENEMY_REWARDS["Boss"])
    }

    h, s, c, v = stats[type_name]

    # Find an dead enemy in the pool and recycle it
    for enemy in self.pool:
        if not enemy.alive:
            enemy.reset(start_pos[0], start_pos[1], h, s, c, v, type_name)
            return enemy

    # If pool is full, expand it (Fallback)
    print("Warning: Object Pool exhausted. Expanding pool.")
    new_enemy = Enemy(start_pos[0], start_pos[1], h, s, c, v, type_name)
    self.pool.append(new_enemy)
    return new_enemy

# Create a global instance of the pool
enemy_pool = EnemyPool()

class TowerFactory:
    """Implements the Simple Factory pattern to create different tower types."""
    @staticmethod
    def create_tower(type_name, x, y):
        if type_name == "Basic":
            return BasicTower(x, y)
        elif type_name == "Splash":
            return SplashTower(x, y)
        elif type_name == "Slow":
            return SlowTower(x, y)
        elif type_name == "Sniper":
            return SniperTower(x, y)
        elif type_name == "Rapid":
            return RapidTower(x, y)
        else:
            raise ValueError(f"Unknown tower type: {type_name}")

```

Appendix D: Headless Automated Regression Testing

```

import os
import unittest

# --- HEADLESS MODE ---
# This tells PyGame to run without opening a graphical window.
os.environ["SDL_VIDEODRIVER"] = "dummy"

import pygame
pygame.init()

```

```

# Import your game modules
import settings
from entities import Enemy, BasicTower
from game_manager import Game

class TestGameStability(unittest.TestCase):

    def setUp(self):
        """This runs before EVERY test to ensure a clean state."""
        # Provide dummy screen dimensions and paths so entities don't crash
        settings.SCREEN_WIDTH = 800
        settings.SCREEN_HEIGHT = 600
        settings.GAME_AREA_WIDTH = 600
        settings.PATH_POINTS = [(0, 0), (100, 0)]

    # --- TEST 1: Enemy Damage & Armour Logic ---
    def test_armoured_enemy_damage(self):
        """Tests that 'Armoured' enemies correctly reduce incoming damage by 1."""
        # Create an Armoured enemy with 8 Health
        armoured_enemy = Enemy(0, 0, health=8, speed=2, colour=(0,0,0), value=20,
type_name="Armoured")

        # Hit it for 3 damage. Because it is Armoured, it should only take 2 damage.
        armoured_enemy.take_damage(3)

        # 8 initial health - 2 actual damage = 6 remaining health
        self.assertEqual(armoured_enemy.health, 6, "Armoured enemy did not calculate damage
reduction correctly.")

    def test_basic_enemy_death(self):
        """Tests that a basic enemy dies when health reaches 0."""
        basic_enemy = Enemy(0, 0, health=3, speed=2, colour=(0,0,0), value=15,
type_name="Basic")
        basic_enemy.take_damage(3)

        self.assertEqual(basic_enemy.health, 0)
        self.assertFalse(basic_enemy.alive, "Enemy should be marked as not alive when
health is 0.")

    # --- TEST 2: Tower Targeting System ---
    def test_tower_targeting_closest_enemy(self):
        """Tests that towers correctly identify enemies within their range."""
        # Place a tower at coordinates (50, 50)
        tower = BasicTower(50, 50)
        tower.attack_range = 100

        # Enemy 1 is at (50, 100) -> Distance is 50 pixels (IN RANGE)
        enemy_in_range = Enemy(50, 100, health=10, speed=1, colour=(0,0,0), value=10,
type_name="Basic")

        # Enemy 2 is at (250, 50) -> Distance is 200 pixels (OUT OF RANGE)
        enemy_out_of_range = Enemy(250, 50, health=10, speed=1, colour=(0,0,0), value=10,
type_name="Basic")

        # Pass both enemies to the tower's update loop

```

```

        tower.update([enemy_in_range, enemy_out_of_range])

        # The tower should have locked onto the enemy in range
        self.assertEqual(tower.target, enemy_in_range, "Tower failed to target the enemy
within its range.")

# --- TEST 3: State Manager Transitions ---
def test_game_state_transition(self):
    """Tests that the GameManager transitions between states without crashing."""
    dummy_screen = pygame.Surface((800, 600))
    game = Game(dummy_screen)

    # The game should initialise in the MainMenuState
    self.assertEqual(game.active_state.__class__.__name__, "MainMenuState", "Game did
not start in MainMenuState.")

    # Tell the manager to switch to GameplayState
    game.set_state("gameplay")

    # The active state should now be updated
    self.assertEqual(game.active_state.__class__.__name__, "GameplayState", "Game
failed to transition to GameplayState.")

if __name__ == '__main__':
    unittest.main()

```

Appendix E: Deployment, Video Demonstration, and User Manual

Video Demonstration A full video demonstration explaining the architecture, GIL circumvention, and gameplay features can be viewed here:

https://drive.google.com/file/d/1PW9dbjsXXzKlHmPXEEPd6HcH2Y8lfCZW/view?usp=drive_link

Environmental Requirements

- Python 3.12 or higher
- PyGame 2.5.2 or higher

Installation and Execution

1. Extract the submitted zip file or clone the repository.
2. Open a terminal and install the required dependencies using: `pip install pygame`
3. To launch the game, execute the main script: `python3 main.py`
4. To run the automated headless regression tests, execute: `python3 -m unittest tests/test_logic.py`